

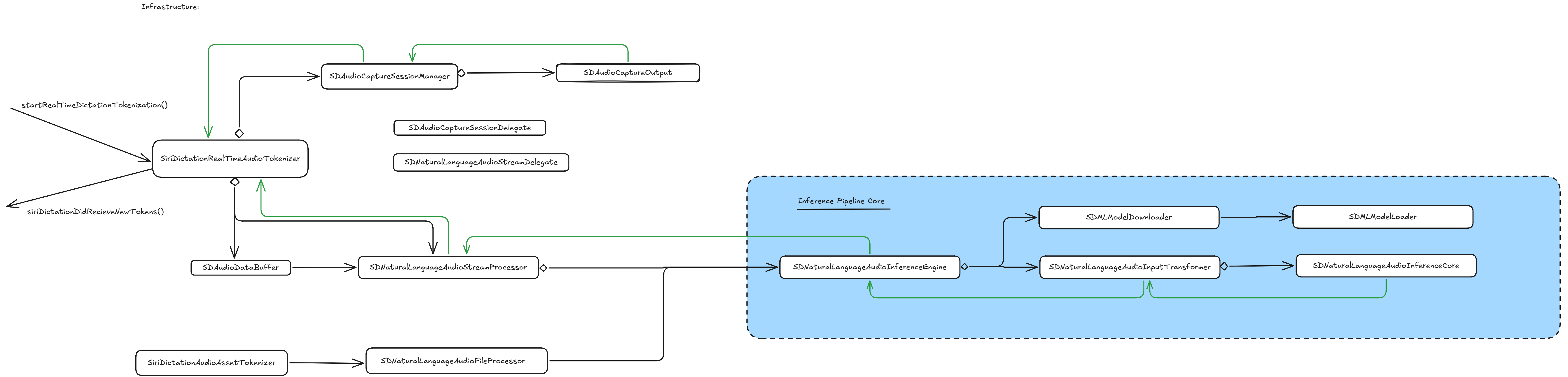
Siri Dictation Design

Functional Requirements:

- Build Siri Dictation Feature, as a user talks into a microphone the feature should translate the speech into text into real-time
- We want to integrate this feature into UI Components like the ios Keyboard
- A simple microphone button should be the entry point into the Siri Dictation experience.
- On user tap this should kick off Siri Dictation feature.
- If user taps again it should turn it off.

Non-Functional Requirements:

- Latency is very important for this product, the time it takes for users to get text output after speaking into the microphone should be minimal. Voice should feel like its getting translated in real-time.
- The infrastructure should be real-time, it should be able to handle streams
- Optimize for keeping the main thread free as user's will be typing, UX needs to be responsive:
- Input handling should be responsive
- Streaming/updating UI with text tokens should also be done in a efficient, responsive manner.
- Design should be scalable. As a v1 we may integrate this into a KeyBoard style experience, but as a v2 we may integrate this into a totally different UI interface.



Data Flow:

1. Product calls the SiriDictationRealTimeAudioTokenizer and says startTokenizing(), and supplies a delegate to handle the processed tokens.
2. SiriDictationRealTimeAudioTokenizer first established a audio capture session/pipe by first calling startAudioCaptureSession() on SDAudioCaptureSessionManager.
3. SDAudioCaptureSessionManager creates captureSession with the input device as the iPhone microphone and sets a custom SDAudioCaptureOutput object to handle the audio data (output/sink of the pipe). Leverages AVSession.
4. The SDAudioCaptureOutput receives the raw audio data (CMSampleBuffer) on a background serial queue (set up by the manager).
5. The audio bytes from the CMSampleBuffer are quickly returned to the SessionManager and then back to the SiriDictationRealTimeAudioTokenizer over Delegate API.
6. SiriDictationRealTimeAudioTokenizer recieves the bytes and quickly buffers them, storing them in a SDAudioDataBuffer.
7. After storing the bytes, SiriDictationRealTimeAudioTokenizer calls the SDNaturalLanguageAudioStreamProcessor to process the data bytes. It kicks off the API call async on another background serial queue dedicated for the SDNaturalLanguageAudioStreamProcessor, this keeps the audio buffering "path" light-weight, just recieve bytes and buffer on the SDAudioCaptureOutput thread context.
8. SDNaturalLanguageAudioStreamProcessor gets a request to process new bytes, first it reads a window of *N* bytes from the SDAudioDataBuffer and passes the data alone to the SDNaturalLanguageAudioInferenceEngine directly. Remember now this is running on its own background serial queue thread context.
9. SDNaturalLanguageAudioInferenceEngine is just a top-level facade for the inference pipeline. It directly passes the bytes down to the SDNaturalLanguageAudioInputTransformer
10. SDNaturalLanguageAudioInputTransformer then transforms the bytes into a data/tensor format understood by the ML Model and then passes this to SDNaturalLanguageAudioInferenceCore
11. SDNaturalLanguageAudioInferenceCore then performs the ML inference and we return the output back up each layer all the way to the top product API

Rough Product Interface Sketch:

```
import Foundation

public typealias SessionCompletion = ((Bool, Error?) -> Void)
public protocol SiriDictationAudioCaptureSessionManager {
    func startAudioCaptureSession(_ callbackQueue: DispatchQueue, _ completion: SessionCompletion)
    func stopAudioCaptureSession(_ completion: SessionCompletion)
}

public protocol SiriDictationTokenStreamDelegate {
    func siriDictationDidRecieveNewTokens(_ tokens: [String])
}

/*
 * Very simple API interface, Client.
 * Start real-time dictation tokenization collection, and stop.
 * The client has no idea about the session management and inference
 * pipeline under the hood. Clients get real time processed words
 * via the SiriDictationStreamDelegate
 */
public class SiriDictationRealTimeTokenizer {

    public init(_ tokenDelegate: SiriDictationTokenStreamDelegate,
              _ audioSessionManager: SiriDictationAudioCaptureSessionManager) {}

    public func startRealTimeDictationTokenization(_ callbackQueue: DispatchQueue) -> Void {}

    public func stopRealTimeDictationTokenization() -> Void {}
}

/*
 * Clients will use a factory to create the real time tokenizer.
 * Simplifies creation of the tokenizer for the client because we will
 * inject all dependencies via the constructor to enable high amount of
 * extensibility to swap different implementations of the core.
 */
public struct SiriDictationRealTimeTokenizerFactory {

    public static func createDefaultRealTimeTokenizer() -> SiriDictationRealTimeTokenizer? {}
}
```

Things to think through:

- How do you handle natural language translation. A request pattern could be good.
- performance metric for the latency

Dad's thoughts/feedback:

1. Diagrams of the UI would have been helpful because its useful to see the endpoint and start to the system. Even a one-liner is fine.
2. Locking and mutexing the Buffer to have it be thread-safe is the proper call/direction.
3. Dad said try to keep the buffer uncapped, its on your system to read data under a performance/latency metric guarantee. If you need to cap instead of circular buffer just add a cap to the queue and dont accept new data.
4. A good rule of thumb is if you wake up a thread there should be work to do.

ChatGPT Feedback:

1. There are "lock-free" ring buffers. Apparently these are a lot more performant in consumer/producer scenarios and its not good to lock when acquiring new AVAudioDataOutput.
2. On scenario that ChatGPT flagged is audio data may come in small bytes and quite frequently, so everytime we add to the buffer and dispatch to read can be inefficient (write storm, enqueueing lots of reads). ChatGPT says its better to enqueue *N* bytes, then only after enqueueing *B* bytes dispatch_async an operation to read from our AudioBuffer. This way we process *N* bytes at a time and prevent a read/write storm. A thread-safe byteCount will need to be tracked, and also when we dispatch a read we need to set a flag to true. Wait for that to read to complete, then set the flag back to false. So thread-safe flag needed. This is because when we dispatch a read we want that read to complete first, otherwise we might enqueue a bunch of subsequent reads if more audio data comes in, kinda creates the read storm again.
- Because we can't lock the queue on the producer path even acquiring locks to check the count and flag on the produce path seems dangerous apparently. Swift is recommending to use an Atomic CAS operation instead, can check it out.
- It does feel like best way to handle this is honestly through a DispatchSem.

3. I think the consumer and producer is a good example to use a DispatchSemaphore. Could be a good design problem.